
Beyond Energies and Forces: OMol-Descriptors-4M, an Open Post-DFT Pipeline and Benchmark for Machine Learning in Chemistry

Santiago Vargas

Chemical Science Division
Lawrence Berkeley National Laboratory
SantiagoVargas@lbl.gov

Samuel M. Blau

Energy Technologies Area
Lawrence Berkeley National Laboratory
smbrau@lbl.gov

Abstract

We present generator, an end-to-end computational chemistry post-analysis framework that processes repositories of molecular DFT wavefunctions and yields six partial-charge schemes (Hirshfeld, CM5, ADCH, Becke, Mulliken, Loewdin), four bond-order schemes, full QTAIM topology, fuzzy descriptors, and ORCA-derived globals. We built this by wrapping per-tier analysis routines around Multiwfn and ORCA’s orca_2mkl utility, packaging the result into a scalable workflow for HPC clusters. Users can select compute tiers to trade fidelity for cost, then compact the resulting calculations into portable sharded Lightning Memory-Mapped Databases (LMDBs) keyed by job identity for downstream analysis and machine learning. To demonstrate the pipeline, we processed the recently-released OMol25 electronic structure dataset: a 4M-structure public subset of the 110M+-calculation 2025 release that ships the complete DFT outputs, including density matrices, energy gradients, ORCA .out files, and .gbw wavefunctions. By post-processing the wavefunctions, we extract atom-, bond-, and molecule-level descriptors across ~4M structures spanning 34 chemical verticals, supplementing the original dataset’s energy and force labels with physically-interpretable quantities. We name this dataset OMol-Descriptors-4M and present it as the first multi-vertical descriptor dataset at this scale beyond energies and forces. Analogous to the source dataset, we establish an evaluation protocol with explicit evaluative claims and scope boundaries: composition-ordered train/val/test splits via blake2b-hashed molecular formula, five domain-specific held-out stress tests (metal-ligand bond pairs, lanthanide-ligand bond pairs, reactivity, large systems, and high-charge regimes; ~60k structures total), and quantified per-descriptor noise floors derived from cross-method agreement on identical structures. Together with the open generation pipeline, these components let researchers reuse the protocol on new wavefunction repositories across molecular chemistry.

1 Introduction

Machine learning, and in more recent years deep learning, have completely upended the relationship between large datasets and how we interpret, predict, and generate information within dataset domains. Prior to this revolution, humans relied on low-dimensional embeddings and descriptors to parse and understand large swaths of data. Today, a wide spectrum of models operate on language[cite], vision[cite], and biological [cite] data. In the chemical sciences, machine learning has

33 2 Related Work

34 3 Dataset

35 3.1 Provenance

36 3.2 Descriptor Families

37 3.3 Per-Vertical Breakdown

38 3.4 Storage Layout

39 3.5 Data Quality

40 4 Pipeline as Contribution

41 4.1 Architecture

42 The workflow pipeline is an orchestration layer over two external binaries, ORCA's `orca_2mk1` Neese
43 [2012] and Multiwfn Lu and Chen [2012]. The OMol25 release initially focused on energies and
44 forces with a limited set of properties in their `aselmdb` metadata. They subsequently released the
45 4M electronic structure dataset hosted at Argonne National Lab with more complete coverage of
46 electronic properties. The expanded 4M segment includes a slew of additional information including
47 wavefunctions (as `gbws`), density matrices, and the raw outputs from the finalized ORCA calculations
48 (`.out`, `.engrad`, `orca_property.txt` files, etc.). This 4M release is coupled to our post-processing
49 pipeline to yield an even richer set of descriptors for the community to train models and evaluate
50 beyond the standard gamut of energies and forces for machine-learned interatomic potentials (MLIPs).

51 Our pipeline reads in the `orca.out` and wavefunction file from each OMol job folder to commence.
52 The `gbw`, in particular, is the foundation for all downstream analysis. As Multiwfn has more complete
53 compatibility with `.wfn/.wfx` files, the first step the pipeline performs is a Molden conversion
54 through `orca_2mk1` followed by EDF library loading inside Multiwfn so that ECPs are considered in
55 downstream analyses. Without this EDF loading, there are potential inconsistencies with how ECPs
56 are treated **TODO: cite multiwfn manual where it says that**. Folders that fail the EDF load are flagged
57 automatically (see Sec. 3) and re-queued.

58 The user selects a calculation tier through a single integer, `full_set` $\in \{0, 1, 2\}$, which cascades
59 through the four post-DFT families implemented in `qtaim_gen/source/data/multiwfn.py`. Tier
60 0 is the cheapest baseline with four partial charge schemes (Hirshfeld, ADCH, CM5, Becke), one
61 bonding scheme (fuzzy bond order), full QTAIM analysis, and Becke/Hirshfeld fuzzy density with
62 optional spin variants. Tier 1 adds three more charge schemes (VDD, MBIS, CHELPG), IBSI bond
63 order, and ELF + MBIS fuzzy density. Tier 2 adds Bader (basin-integrated) charges, Laplacian
64 bond orders, and Laplacian and gradient-norm fuzzy densities. Tier 2 enables the more expensive
65 electrostatic potential geometry analysis (binned into our other group of calculations). The tiers are
66 subsets of each other, this signifies that a user can "stack" calculations from 0 to 2 to incrementally
67 add richer descriptors as they go. The tiered design is intended to give users a choice of how much
68 compute they want to invest in the post-processing step, while allowing for future expansions. Table 1
69 summarizes the per-family contents of each tier; the QTAIM critical-point search is run at every tier
70 and the ORCA-derived globals are always parsed.

71 Every job folder carries its own state of completed calculations and whether it is being worked on.
72 A `.processing.lock` file with a stale-mtime threshold protects against conflicting work across
73 distributed workers, and a per-folder `out_files.zip` archives every Multiwfn `.out` stream so a folder
74 can be reparsed or analyzed without re-running conversions or Multiwfn. Validation is structural
75 rather than numeric: each tier defines the JSON keys that must be present, and the pre-validation pass
76 treats a folder as done only if all expected keys are populated. ECP-failed folders are detected by
77 scanning the archived ADCH/CM5 logs for the `Loading EDF library finished` banner.

78 The per-folder outputs are seven JSON files: `charge.json`, `bond.json`, `qtaim.json`,
79 `fuzzy_full.json`, `other.json`, `orca.json`, and `timings.json`. The first five are Multiwfn-
80 derived and indexed by method name. The full per-method schema for each tier is enu-

Table 1: Per-family contents of each calculation tier (`full_set`). Tiers are strict supersets: tier k runs everything in tiers $0, \dots, k$. Schemes shown in a column are those *added* at that tier. Source: `qtaim_gen/source/data/multiwfn.py`.

Family	Tier 0 (baseline)	Tier 1 (+adds)	Tier 2 (+adds)
Partial charges	Hirshfeld, ADCH, CM5, Becke	VDD, MBIS, CHELPG	Bader (basin-integrated)
Bond orders	Fuzzy bond order	IBSI bond order	Laplacian bond order
Fuzzy density	Becke density, Hirshfeld density, Becke spin, Hirshfeld spin	ELF, MBIS density, MBIS spin	Laplacian ρ , gradient-norm $ \nabla\rho $
Other (scalar fields)	Geometry (ellipticity), ALIE	—	ESP geometry

Always-on (run at every tier): full QTAIM critical-point search (atoms, bond CPs, ring CPs, cage CPs; ρ , $\nabla^2\rho$, ellipticity, energy density); ORCA .out parsed globals (energies, orbital eigenvalues, dipole, gradient, Mulliken / Loewdin charges, SCF metadata, quality flags). Note: ORCA’s Mayer population analysis block reports its QA column as “Mulliken gross atomic charge”; this is stored for completeness but is not treated as an independent charge scheme in downstream analysis.

merated in Sec. 3; we only sketch the families here. `orca.json` is parsed directly from the ORCA .out file (energies, orbital eigenvalues, dipole, gradient, SCF metadata, Mulliken/Loewdin charges, and quality-filter fields) by a single-pass enum state machine implemented in `qtaim_gen/source/core/parse_orca.py`. We chose this over a general parser like `cclib` because the state machine streams the file once, has zero runtime dependencies beyond the standard library, and only extracts the fields we need. `timings.json` stores per-step wallclock seconds and is carried through to LMDb as a provenance record so every downstream artifact can be audited back to its compute cost. We do note that `timings.json` does not store number of cores, architectures, or other details for exact comparisons but is still useful for relative analysis.

Once the desired tier is complete, the JSON outputs are converted into a set of typed LMDb files via `json-to-lmdb`: one LMDb per descriptor family (geometry, charge, bond, qtaim, fuzzy, other, orca, timings) sharing a common key derived from the relative path of the job folder. The compact LMDb representation is portable and self-describing, so it is directly usable by external graph models such as Chemprop **TODO: cite** or any custom PyG/DGL pipeline. For users of our companion package `qtaim_embed`, the `generator-to-embed` entrypoint composes the typed LMDbs into PyG heterographs through the `GeneralConverter` class, with configurable bonding schemes (structural cutoff, fuzzy, IBSI, or QTAIM), charge filter, fuzzy filter, and ORCA-globals filter. A second `QTAIMConverter` class is retained specifically for QTAIM-only graphs, where bond edges are defined by QTAIM bond critical points rather than by structural cutoffs. The resulting graphs expose atom-level, bond-level, and global-level targets simultaneously.

Foundation for downstream ML work. This pipeline is intended to provide not just a means to automate high-throughput DFT analysis, but also to be a stable foundation for scalable downstream ML work. LMDbs were used throughout, with matching keys across all descriptor families, to provide a rapid source for data factories in ML models. Researchers can leverage this breathe of available information to train models for partial charges, bond orders, teacher-student models for downstream tasks, foundational models, etc. With the converters for `qtaim_embed`, we also provide a working method for training graph neural networks (GNNs) **TODO: cite** including a diverse set of message-passing and global-readout layers, more detail on that in Sec. 4.3.

Figure F1 prompt (for image generation). The following prompt can be passed to a vector-capable image generator (Nano Banana, Claude, or similar) to render Figure F1:

Generate a clean horizontal SVG flow diagram of a quantum chemistry post-processing pipeline. Left to right, four stages connected by labeled arrows. Stage 1 (input): a single block labeled "OMol25 .gbw" with a small sub-arrow into "orca_2mkl → Molden + EDF". Stage 2 (compute, drawn as a stack of three escalating boxes labeled "Tier 0: 4 charges, fuzzy bond, QTAIM, baseline fuzzy density", "Tier 1: +VDD, +MBIS, +CHELPG, +IBSI, +ELF/MBIS fuzzy", "Tier 2: +Bader, +Laplacian bond, +Laplacian/∇ρ fuzzy, ESP geometry"), with two side annotations: a small lock icon labeled ".processing.lock

(distributed safety)” and a zip icon labeled “`out_files.zip` (reparse without recompute)”. Stage 3 (per-folder JSON outputs): seven small file icons in a row labeled `charge.json`, `bond.json`, `qtaim.json`, `fuzzy_full.json`, `other.json`, `orca.json`, `timings.json`. Stage 4 (LMDb + graphs): a database cylinder labeled “Typed LMDbs (one per family, shared keys)” followed by two output branches, one labeled “Portable: Chemprop / custom PyG” and one labeled “`qtaim_embed` → DGL heterographs (atom / bond / global tasks)”. Use a muted blue and gray palette, sans-serif labels, no decorative icons beyond the stated ones.

Features flagged for trimming before camera-ready. While auditing the codebase to write this section, two threads of dead or near-dead capability surfaced. First, `qtaim_gen/source/core/parse_critic.py` (Critic2 critical-point parser) is no longer imported by any active workflow; the only remaining reference is an optional `critic2_cmd` field in `create_files.py` that gates a legacy reaction workflow. We intend to drop the Critic2 dependency entirely so that Multiwfn is the sole QTAIM backend. Second, the older QTAIM-only job runners under `scripts/old/` (`generator_run.py` and the QTAIM-specific Parsl runner variants) predate the multi-method full-runner family and can be removed without loss of capability. We retain QTAIMConverter itself because it underpins the bond-classification task.

4.2 Sharded Execution and LMDb Merge

LMDb is single-writer and exhibits lock contention under naive parallel writes, so `json-to-lmdb` does not attempt to share a single output environment across workers. Instead, each parallel job is given a shard index $i \in \{0, \dots, N - 1\}$ and a total shard count N , and partitions the input folder list by index modulo N . For the flat `root/category/subset/job` hierarchy this is a glob plus a modulo filter; for jagged hierarchies such as OMol4M, where job folders sit at variable depth, the user supplies an explicit `-folder_list` file of absolute paths and the same modulo partitioning applies to the line index. In both cases the LMDb key is `relpath(folder, root_dir).replace(os.sep, "__")`, which is identical across every descriptor family, so the per-family shards always share keys and joining is trivial.

Each shard writes into its own subdirectory as `shard_{i}.lmdb`. When the last shard finishes its work it drops a sentinel file (`shard_N.done`); the merge step waits on the full sentinel set and then concatenates per-family shards into a single consolidated LMDb per family. The same shard/merge pattern is implemented at the converter stage in `converter.py`, so graph construction can also be parallelized and joined into one final per-vertical graph LMDb.

4.3 ML-Ready Post-Processing

The LMDb stage is the boundary between the pipeline and downstream machine-learning use. Three features sit on top of that boundary and are intended to make benchmark construction turnkey rather than reinventable.

The first is *merging*: after a sharded run, the per-shard LMDbs across all descriptor families are merged into one LMDb per family with a single pass that reads each shard, copies records, and verifies key uniqueness. Merge is the operation that produces the artifact a benchmark consumer actually reads.

The second is *multi-vertical splitting*. The `multi-vertical-merge` entrypoint takes a pipeline JSON config that lists multiple verticals (e.g. SPICE, QM9, RMechDB, OMol25 subsets) and produces per-vertical train/val/test graph LMDbs whose splits are globally consistent: a molecular formula that appears in two verticals is assigned to the same split partition in both, so a held-out composition is held out everywhere. The implementation lives in `qtaim_gen/source/utils/multi_vertical.py` and runs in three phases (Plan, Build, Scale).

The third is *train-only scaler fitting*. Feature scalers (atomic, bond, and global) are fit on the train split only and applied to validation and test, eliminating the quiet-but-common leakage path where a scaler fit on the full pool implicitly tells the model the test-set mean. When `multi-vertical-merge` is used, the scaler is fit on the union of all train splits across verticals and then applied per-vertical, which is the right behavior when the downstream model is trained on the full mixture.

169 Together these three features turn a raw LMDB dump into something a benchmarking team can load
170 directly: composition-consistent splits, per-vertical files, leakage-safe scalers, and deterministic keys.

171 4.4 HPC Portability

172 We exercised the pipeline on three production HPC centers, each with a different sched-
173 uler and a different parallelism paradigm. ALCF Polaris uses PBS Pro and a controller-and-
174 workers Parsl topology: one login or batch node hosts the Parsl controller, which farms indi-
175 vidual job folders out to PBS-allocated worker nodes through the PBSProProvider executor in
176 `qtaim_gen/source/utis/parsl_configs.py`. NERSC Perlmutter uses Slurm and the same
177 topology with the SlurmProvider executor; the only meaningful change is the queue name and the
178 per-node thread budget. Both centers scaled cleanly because the controller node sees only the work
179 queue and the worker nodes see only their assigned folders.

180 LLNL Tuolumne uses Flux as its native scheduler, and we did not get the controller-and-workers
181 topology to behave reliably under Flux. The fallback used on Tuolumne was the single-node paradigm:
182 a Slurm or Flux allocation grants a node, and a local Parsl ThreadPoolExecutor chews through the
183 local job queue with no remote dispatch. This is the same path used for laptop and workstation runs
184 (full-runner), parameterized by `n_workers` and `n_threads_per_job`. NERSC and ALCF can
185 also be run in this single-node mode if a user prefers not to manage a controller, at the cost of losing
186 multi-node scaling.

187 The submission scripts and Parsl configs for all three centers are checked in under
188 `qtaim_gen/source/utis/parsl_configs.py`; PBS, Slurm, and the single-node Flux fallback
189 are all minimal edits of the same template. Reproducing a run on a new center is expected to require
190 swapping the provider class and the queue/account fields, nothing more.

191 4.5 Throughput and Parallel Scaling

192 Here we report a local scaling experiment where we intent to show the concurrency performance of
193 Multiwfn on our routines. The local sweep is run on 16 copies of a moderately-sized structure drawn
194 from the dataset (from the droplet vertical here). It contains 36 atoms where we ran the three levels of
195 calculations in triplicate with Multiwfn 3.8. We sweep the number of concurrent job-folder workers
196 $\in \{1, 2, 4, 16\}$ at a fixed budget of cores (16) running to unearth how concurrency impacts wallclock.
197 The configurations are W16T1 (16 workers $\times$ 1 thread/job), W4T4 (4 workers $\times$ 4 threads/job),
198 W2T8 (2 workers $\times$ 8 threads/job), and W1T16 (1 worker $\times$ 16 threads/job). Here we see that
199 single-threaded jobs yield the best performance with lower wallclock times across all three calculation
200 levels. Thus, we recommend users to prioritize more workers with fewer threads per job assuming
201 that you do not run into memory bottlenecks on larger systems with more memory-intensive routines.

202 4.6 Reproducibility

203 The pipeline pins no specific version of ORCA or Multiwfn at runtime; some compatibility across
204 versions of either is expected though we have not exhaustively tested versions of each. For example,
205 we also tested Orca5~~TODO: cite~~ wfn conversion where our code handles its treatment of ECPs and as
206 well as Orca 6.0 and 6.1. In the case of Multiwfn, any version that follows the same menu structure
207 should work though we only tested 3.9 and 3.8. The dataset reported here was produced with ORCA
208 6.0.0, the same release used for OMol25 and Multiwfn 3.8. In our experience, the `orca_2mkl` version
209 should match that of the gbws from input DFT calculations.

210 Determinism in the pipeline is structural rather than numeric: the LMDB key derivation, the per-tier
211 set of expected JSON keys, and the parsing logic for both Multiwfn and ORCA outputs are fixed by
212 code and covered by the test suite under `tests/`. Numeric reproducibility of the underlying QM
213 calculation is the responsibility of ORCA itself and is not something we modify.

214 The test suite serves as the end-to-end smoke test: `tests/test_lmdb.py` and
215 `tests/test_general_converter_filters.py` together exercise json-to-lmdb, sharded
216 conversion, merge, the GeneralConverter graph construction, and scaler fit/apply on a small fixture set
217 under `tests/test_files/lmdb_tests/`. Running `pytest -q` on a fresh checkout is therefore a
218 ground-truth check that a user's local toolchain produces the same LMDB and graph artifacts we
219 ship.

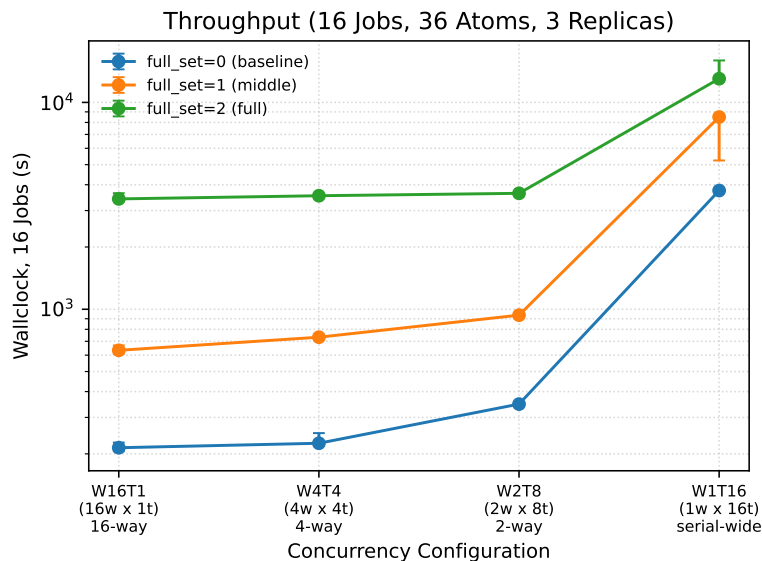


Figure 1: Multiwfn QTAIM post-processing wallclock as a function of intra-node concurrency configuration. Each configuration processes 16 homogeneous copies of `droplet_test` (36 atoms, def2-TZVPD) under cascade-restart tiers. Configurations saturate a 16-thread budget differently: W16T1 (16 workers $\times$ 1 thread), W4T4 (4 workers $\times$ 4 threads), W2T8 (2 workers $\times$ 8 threads), W1T16 (1 worker $\times$ 16 threads). Each curve represents a calculation tier (`full_set` 0=baseline, 1=middle, 2=full). Tiers 1 and 2 use `-restart` from the prior tier, so per-tier totals are cumulative phase wallclocks. Markers show medians across 3 replicates with min/max whisker error bars. Y-axis is logarithmic to visualize the $75\times$ range spanning W16T1 tier 0 (fastest) to W1T16 tier 2 (slowest). Hardware: AMD 7950X (16c/32t). ORCA 6.0.1 (`shared_openmpi416`) with system OpenMPI 4.1.2; Multiwfn 3.8 (Oct 2024 dev). ORCA SCF is precomputed; this measures post-processing only.

220 5 Coverage vs Prior Descriptor Datasets

221 5.1 Framing

222 5.2 Method

223 5.3 Results

224 6 Cross-Method Noise Analysis

225 6.1 Methodology

226 6.2 Charge-Method Comparison

227 6.3 Bond-Order Comparison

228 6.4 QTAIM Internal Redundancy

229 6.5 Per-Vertical Noise Floors

230 6.6 High-Disagreement Chemistry

231 7 Census

232 OMol-Descriptors-4M comprises 3,985,130 structures across 34 chemical verticals, indexing
 233 1,302,074 unique alphabetical formulas. At the descriptor level the corpus carries 218,519,775
 234 atom-level records, 30,203,025 charge-scheme records (per-job, per-method), 253,974,706 QTAIM

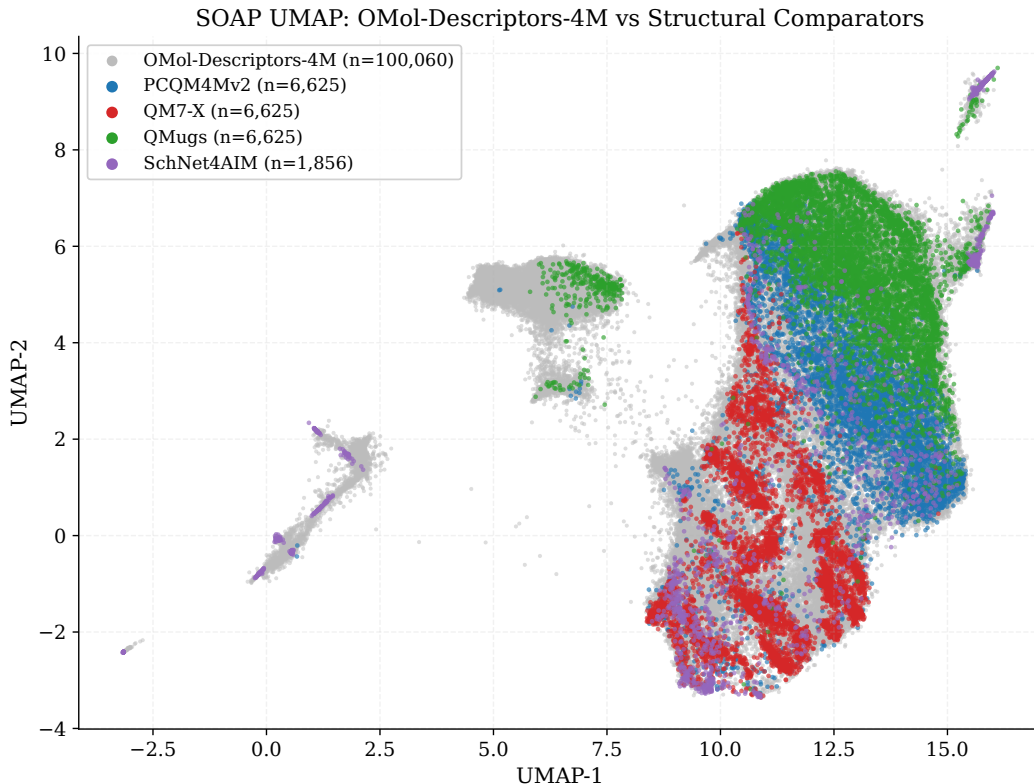


Figure 2: SOAP-UMAP embedding of OMol-Descriptors-4M against four structural comparator datasets. Each point is a molecule projected via UMAP fitted on a 500-molecule-per-source balanced subsample (2,500 anchor points total). SOAP descriptors use a 12-element species set (H, B, C, N, O, F, Si, P, S, Cl, Br, I) common to all five sources. OMol-Descriptors-4M (gray) is rendered as a background haze to expose the relative positions of PCQM4Mv2, QMugs, QM7-X, and SchNet4AIM.

235 bond critical points, and 1.58 billion bond-order records spanning the three universal schemes. All
 236 counts in this section are corpus-level totals prior to the train/val/test partition introduced in Sec. D.3.

237 7.1 Bond records per vertical

238 Figure 3 shows the per-vertical bond record count, decomposed by scheme. Three bond-order
 239 schemes are reported on every structure: Mayer (ORCA), Löwdin (ORCA), and fuzzy bond order
 240 (Multiwfn). At the corpus level Löwdin contributes the largest share at 688,904,594 records (43.6%),
 241 followed by fuzzy at 491,385,480 (31.1%) and Mayer at 401,137,813 (25.4%). Each row in the
 242 stacked bar therefore corresponds to three independent estimates of the same underlying bond, not
 243 three distinct bonds; cross-scheme agreement on those shared bonds is treated separately in Sec. 6.

244 The vertical-level distribution is heavy-tailed. The omol vertical alone accounts for 680,873,135
 245 records (43% of the bond corpus); the two pdb_fragments verticals together contribute
 246 159,038,457; the DFT-relaxed geom_orca6 subset adds 76,494,584. The lower end is set by
 247 noble_gas_compounds (1,404 records, 18 structures), which we retain as a stress case for sparse-
 248 bonding regimes. Scheme mix is also non-uniform across verticals. electrolytes_scaled_sep
 249 is Mayer-heavy (46% of its bond records), reflecting the polar bond profile of the scaled-separation
 250 geometries; rgd_uks is Löwdin-dominant (44%).

251 7.2 Charge-scheme records per vertical

252 Charge-scheme records track how many partial-charge methods executed successfully on each
 253 structure. The corpus carries 30,203,025 records across 3,985,130 structures, an average of 7.58

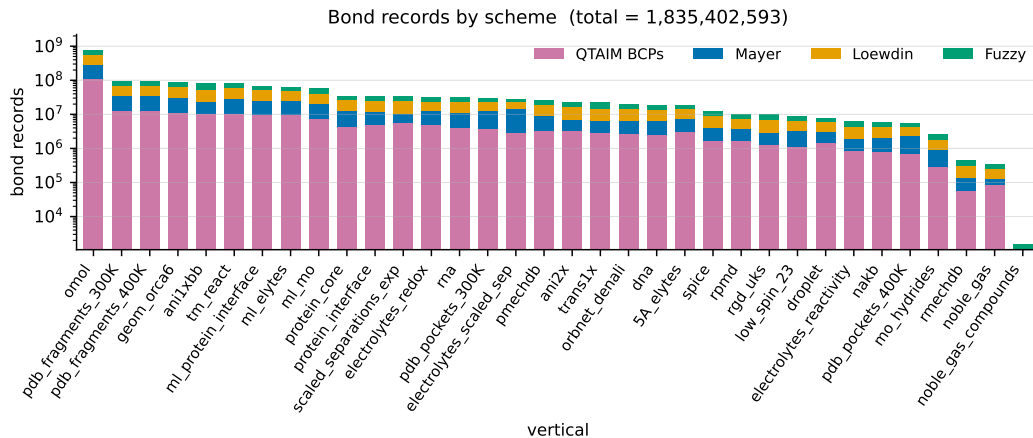


Figure 3: Per-vertical bond-order record counts, stacked by scheme (Mayer, Löwdin, fuzzy) on a log axis. Verticals are sorted by total bond-record count. The three schemes report on the same set of bonds, so counts represent records, not unique bond instances. Corpus total across the three schemes is 1.58 billion records.

254 schemes per structure, consistent with the six independent methods documented in Sec. 3 plus the
 255 ORCA-side Mulliken cross-check. Per-vertical totals span four orders of magnitude, from `rmechdb`
 256 (28,024 records over 2,803 structures) to `omol` (9,997,947 over 1,428,279). The full per-vertical
 257 breakdown is given in Table 2 in the appendix.

258 Per-element-pair bond-order distributions and pairwise cross-method agreement (charges, dipoles,
 259 bond orders) are reported in Sec. 6; the formula-hashed train/val/test partition built from these counts
 260 is described in Sec. 8.

261 8 Evaluation Protocol

262 8.1 Tasks

- 263 • **T1 – Per-atom partial charge regression.** Multi-target: ADCH, Lowdin, Mulliken, Hirsh-
 264 feld, CM5 jointly per atom. Headline: per-method, per-element MAE and Pearson r on the
 265 test split. Long-tail per-element table in appendix.
- 266 • **T2 – BCP property regression.** Targets: ρ , $\nabla^2\rho$, ellipticity at every QTAIM bond critical
 267 point. Headline: per-element-pair MAE on ρ , Pearson r per pair, stratified by bond type
 268 (single, double, aromatic, metal-ligand) inferred from QTAIM topology.
- 269 • **T3 – Bond classification.** Binary: does a QTAIM BCP exist between an atom pair within
 270 $2 \times \text{sum-of-covalent-radii}$. Compared against Mayer-ORCA / Löwdin-ORCA / fuzzy
 271 agreement. Macro-F1 plus per-element-pair confusion stats.
- 272 • T3 sharpening (open): mine the disagreement subset (QTAIM-bonded vs distance-bonded)
 273 once `bond.lmdb` and `qtaim.lmdb` are populated end-to-end; that subset is the discriminat-
 274 ing slice.

275 8.2 Main Split

- 276 • Composition-ordered, deterministic: each `formula_hill` hashed via BLAKE2b Aumasson
 277 et al. [2013] into $N = 10$ buckets.
- 278 • Buckets 0-7 train, bucket 8 val, bucket 9 test. Holdouts (§8.3) are excluded before bucket
 279 assignment.
- 280 • Realized post-holdout sizes: train 3,189,390 / val 399,690 / test 415,913.
- 281 • Bucket-size variance under 5% of the mean; per-bucket unique-formula count $\approx 112\text{k}$ each,
 282 confirming the partition balances composition and not row count.

- Implementation: `split_descriptor_lmdbs.py` runs per-vertical, sharded over verticals, race-free under parallel xargs; `per-vertical_split_assignment.json` is the durable record.
- Verticals are not sampled uniformly because the split is global by formula. A per-vertical post-split distribution table will be published in the appendix as a transparency artifact.

8.3 Held-Out Evaluation Sets

The held-out sets are constructed independently of the main split and materialized before any model is trained. Membership is decided from the manifest, and from `bond.lmdb` for the two metal-ligand sets, then removed from the train, val, and test buckets so the same structure is never simultaneously evaluated as in-distribution test and as a stress test. Overlaps between held-out sets are allowed (a structure can be both H7-large and H8-high-charge), but each set is materialized once and reported separately so the reader can attribute error to the axis being stressed. None of these sets is used for hyperparameter selection or early stopping; numbers are reported once at the end of training.

The headline generalization signal per set is the gap $\Delta = \text{metric}_{\text{held-out}} - \text{metric}_{\text{main test}}$, reported alongside the absolute number so a reviewer can read the floor (test) and the stress response (Δ) on the same line. A small Δ on H7 means size extrapolates; a large Δ on H1 means TM-ligand chemistry is being memorized at the pair level.

Three of the five sets (H1, H6, H7) inherit the hardest evaluation axes from the original OMol25 paper directly: rare transition-metal-ligand bond pairs, rare lanthanide-ligand bond pairs, and large-system extrapolation. Where OMol25 publishes a hold-out construction, we lift it unchanged onto our manifest rather than redesigning it, both for comparability and because the OMol25 axes were already chosen to stress chemically meaningful failure modes.

Sets and sizes. The five held-out sets together cover roughly 60,719 structures ($\sim 1.5\%$ of the dataset; overlaps allowed):

- **H1 – Metal-ligand pairs (15,030 structures).** Eleven (TM, partner) bond pairs sampled stratified across log-spaced frequency bands with shape (3, 2, 2, 4, 0) and `seed=87`. Selection is driven by *true* bonds (Mayer / fuzzy bond order ≥ 0.3) extracted from `bond.lmdb`, not element co-occurrence in the formula; the full procedure is in Appendix D.1. Tests whether the model can predict descriptors for transition-metal bond chemistry it never saw during training.
- **H3 – Reactivity (12,507 structures).** Composition-stratified subsample of three reactivity-flavored verticals: `tm_react` (5k), `electrolytes_reactivity` (5k), `pmechdb` (2.5k). Per-vertical formulas are chosen by deterministic BLAKE2b hash so a reader can reproduce membership without rerunning the sampler. Tests transfer to non-equilibrium and transition-state-adjacent geometries.
- **H6 – Lanthanide-ligand pairs (2,589 structures).** Same construction as H1 applied to (Ln, partner) bonds; shape (3, 2, 2, 1, 0), `seed=190`; full procedure in Appendix D.2. The set is smaller than H1 because the Ln-bonded subset is itself smaller (66,677 structures vs 651,742) and the highest-frequency band is empty.
- **H7 – Large systems (18,200 structures).** Strict cutoff `n_atoms > 250`. Inherits OMol25’s large-system stress axis unchanged. Targets size extrapolation in the regime where several descriptor methods (notably fuzzy integration and grid-based QTAIM) are most expensive, and where the per-atom signal-to-noise ratio in DFT outputs is hardest to maintain.
- **H8 – Large net charges (12,393 structures).** Strict cutoff `net_charge_abs > 4` (equivalently $|q| \geq 5$). Targets the high-charge tail where partial-charge schemes disagree the most (cf. §6) and where SCF behavior is most sensitive to functional and basis choice.

We deliberately use pair-level held-outs (H1, H6) rather than whole-element held-outs because the goal of this benchmark is to produce models that are usable across the periodic table, not models that have a hole where transition metals or lanthanides should be. Dropping every `has_tm` structure would remove 17.24% of the dataset and evict whole verticals (`low_spin_23`, `tm_react`, `ml_mo`) from training entirely; dropping every `has_lanthanide` structure would remove 1.78% and similarly

empty the Ln-bearing verticals. In both cases the resulting model would simply have no signal to predict on for those elements at inference time. Holding out specific (metal, ligand) pairs keeps every element represented in training while still measuring rare-chemistry transfer, which is the construction the OMol25 paper uses for the same reason.

8.4 Holdout-Aware Splitting Pipeline

- **Step 1 – Pull holdouts.** `pull_holdout_records.py` reads each holdout CSV, materializes per-set LMDBs under `data/holdouts/<set>/`, and emits a `manifest_holdout.parquet` keyed on the LMDb key convention `relpath(folder, root_dir).replace(os.sep, "__")`.
- **Step 2 – Exclude.** `multi_vertical.load_exclusion_set` re-derives LMDb keys from the `manifest_rel_path` column and unions H1/H3/H6/H7/H8 into one per-vertical exclusion set.
- **Step 3 – Split.** `split_descriptor_lmdbs.py` reads `structure.lmdb`, drops excluded keys, assigns each remaining key to train/val/test by `BLAKE2b(formula_hill) % 10`, then partitions each of the 8 descriptor LMDBs (`structure`, `charge`, `qtaim`, `bond`, `fuzzy`, `other`, `orca`, `timings`) into the assigned split.
- Output layout: `<output_dir>/<vertical>/<split>/<descriptor>.lmdb`.
- HPC parallelism: `-include_verticals X + -merge_from DIR`, mirroring the pull pipeline. Per-vertical `split_assignment.json` is the durable, race-free record (no global plan file).
- Lock-step audit: across all 8 descriptors, `source row count = train + val + test + excluded` for every vertical (verified on `ani2x`: `215,642 source = 168,783 + 17,417 + 29,442, n_excluded = 0`).

8.5 Metrics

- Per task: MAE, RMSE, Pearson r .
- **T1** stratifications: per-method, per-element, per-vertical, per-charge-bucket.
- **T2** stratifications: per-element-pair, per-bond-type (covalent / metal-ligand / H-bond), per-vertical.
- **T3** stratifications: macro-F1, per-element-pair confusion, per-distance-bucket recall.
- Held-out sets: same metrics, plus the gap to the corresponding main-test number.

8.6 Methodology and Reporting

- Train on the 8 train buckets. Tune on val (bucket 8). Final numbers reported once on test (bucket 9) and on each held-out set, with no further tuning.
- Held-out sets must not be used for hyperparameter selection or early stopping.
- Reference score-predictions CLI deferred to v2: shipping it for v1 without a calibrated reference model would expose the protocol to garbage-in adoption. v2 will release the script alongside a reference model whose error sits between the cross-method noise floor and naive baselines.
- Cross-method noise floors from §6 are a lower bound on achievable error: any model below the floor is overfitting noise, not learning chemistry. Future work using this benchmark should report the floor alongside model error.
- Per-vertical post-split distribution: a transparency artifact (notebook + appendix table) listing realized train/val/test counts per vertical so readers can weight conclusions by vertical-level coverage.

8.7 Optional Floor Baseline

- **D4** – per-element-pair lookup table emitting the median descriptor value for the matching pair from the train bucket.

381 • Submitted as a floor, never as a baseline. Page-budget contingent.

382 **9 Limitations and Broader Impact**

383 **10 Conclusion**

384 **References**

- 385 Jean-Philippe Aumasson, Samuel Neves, Zooko Wilcox-O’Hearn, and Christian Winnerlein. Blake2:
386 Simpler, smaller, fast as md5. In Michael Jacobson, Michael Locasto, Payman Mohassel, and
387 Reihaneh Safavi-Naini, editors, *Applied Cryptography and Network Security*, pages 119–135,
388 Berlin, Heidelberg, 2013. Springer Berlin Heidelberg. ISBN 978-3-642-38980-1.
- 389 Tian Lu and Feiwu Chen. Multiwfn: A multifunctional wavefunction analyzer. *Journal of Computa-*
390 *tional Chemistry*, 33:580–592, 2012.
- 391 F. Neese. The orca program system. *WIREs Comput. Molec. Sci.*, 2(1):73–78, 2012. doi: 10.1002/
392 wcms.81.

393 A Datasheet for Datasets

394 B License Terms

395 C Reproducibility Checklist

396 D Held-Out Construction Logic

397 D.1 H1 metal-ligand pair selection

- 398 • Element-pair co-occurrence (859 TM pairs in `metal_nonmetal_pairs.csv`) is a coarse
399 upper bound; selection here uses true bonds (Mayer / fuzzy bond order ≥ 0.3) extracted
400 from `bond.lmdb` via `tm_neighbor_lists.py`.
- 401 • Per-vertical bond extraction across 15 TM-bearing verticals; one row per (TM atom, partner
402 atom) bond.
- 403 • Global pair table: **1,101 unique (TM, partner) bond pairs** across **651,742 TM-bonded**
404 **structures** (16.35% of dataset).
- 405 • Frequency bands (log-spaced): B1 [1, 10) 529 pairs; B2 [10, 100) 93; B3 [100, 1k) 233; B4
406 [1k, 10k) 203; B5 [10k, 100k) 43.
- 407 • Stratified sampling shape (3, 2, 2, 4, 0), `numpy.random.default_rng(seed=87)`. B5
408 excluded (Pt-C, Ti-H, etc. dominate training); B4 capped at 4 to keep realized holdout near
409 15k.
- 410 • Selected pairs: *B1* Pt-Na, Rh-Fe, Ag-Pd; *B2* Sc-Ir, Cd-Sb; *B3* Mn-Sb, Ir-Ge; *B4* Tc-Cl,
411 Re-H, Sc-S, Hf-C.
- 412 • Variance: across 30 seeds with shape (3, 2, 2, 4, 0), realized size is $15,227 \pm 4,871$. Seed 87
413 lands within 200 of the 15k target.
- 414 • Reproducibility: `qtaim_gen/source/analysis/build_holdout_csvs.py`; outputs
415 `h1_metal_ligand_pair_definitions.csv` and `h1_metal_ligand_pairs.csv`.

416 D.2 H6 lanthanide-ligand pair selection

- 417 • Same construction as H1, applied to (Ln, partner) bonds via `tm_neighbor_lists.py`
418 `-element_class ln`.
- 419 • Five Ln-bearing verticals: `omol`, `tm_react`, `ml_mo`, `electrolytes_reactivity`,
420 `mo_hydrides`.
- 421 • Global pair table: **453 unique (Ln, partner) bond pairs** across **66,677 Ln-bonded struc-**
422 **tures** (1.67% of dataset). All 15 lanthanides represented.
- 423 • Frequency bands: B1=207, B2=102, B3=86, B4=58, B5=0 (max pair size 4,439). H6
424 effectively samples four bands.
- 425 • Stratified sampling shape (3, 2, 2, 1, 0), `seed=190`. B4 toned down to 1 vs H1's 4 (Ln B4 is
426 sparser, target $\sim 2.5k$).
- 427 • Selected pairs: *B1* Lu-Ni, Ce-La, Gd-Sb; *B2* Ho-F, Yb-Si; *B3* Dy-S, Nd-S; *B4* Tm-N.
- 428 • Reproducibility: same build script as H1; out-
429 puts `h6_lanthanide_ligand_pair_definitions.csv` and
430 `h6_lanthanide_ligand_pairs.csv`.

431 D.3 Holdout-Aware Splitting Pipeline

- 432 • **Step 1 – Pull holdouts.** `pull_holdout_records.py` reads each holdout
433 CSV, materializes per-set LMDBs under `data/holdouts/<set>/`, and emits a
434 `manifest_holdout.parquet` keyed on the LMDB key convention `relpath(folder,`
435 `root_dir).replace(os.sep, "__")`.

- **Step 2 – Exclude.** `multi_vertical.load_exclusion_set` re-derives LMDB keys from the manifest `rel_path` column and unions H1/H3/H6/H7/H8 into one per-vertical exclusion set.
- **Step 3 – Split.** `split_descriptor_lmdbs.py` reads `structure.lmdb`, drops excluded keys, assigns each remaining key to train/val/test by BLAKE2b(`formula_hill`) % 10, then partitions each of the 8 descriptor LMDBs (`structure`, `charge`, `qtaim`, `bond`, `fuzzy`, `other`, `orca`, `timings`) into the assigned split.
- Output layout: `<output_dir>/<vertical>/<split>/<descriptor>.lmdb`.
- HPC parallelism: `-include_verticals X + -merge_from DIR`, mirroring the pull pipeline. Per-vertical `split_assignment.json` is the durable, race-free record (no global plan file).
- Lock-step audit: across all 8 descriptors, `source row count = train + val + test + excluded` for every vertical (verified on `ani2x`: 215,642 source = 168,783 + 17,417 + 29,442, $n_{\text{excluded}} = 0$).

E Per-Method Descriptor Definitions

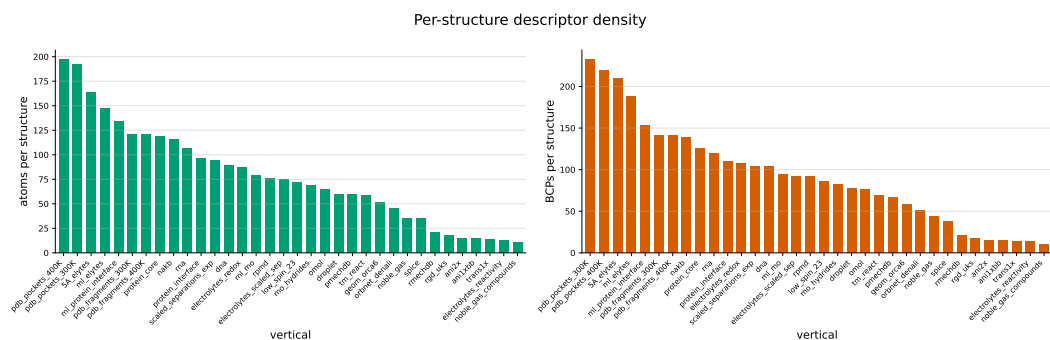
F HPC Configuration Files

G Per-Vertical Census Table

Table 2 expands Sec. 7 into the full per-vertical breakdown (34 verticals plus the corpus total). Figure 4 reports per-structure descriptor density for atoms and QTAIM bond critical points.

Table 2: Per-vertical census of OMol-Descriptors-4M. Counts are corpus-level pre-split totals. *Structs*: structure count. *Forms*: unique alphabetical formulas. *Atoms*: atom-level descriptor records. *Charge rec.*: per-job per-method charge-scheme records. *BCPs*: QTAIM bond critical points. *Mayer / Löwdin / Fuzzy*: per-scheme bond-order records.

Vertical	Structs	Forms	Atoms	Charge rec.	BCPs	Mayer	Löwdin	Fuzzy
5A_elytes	14,200	11,294	2,325,187	99,383	2,982,866	4,474,371	6,655,293	4,244,442
ani1xbb	737,208	48,320	10,834,251	7,318,728	10,801,957	12,465,497	30,634,609	28,118,840
ani2x	215,642	5,773	3,280,277	1,586,066	3,346,555	3,824,490	9,516,073	6,379,670
dna	24,647	4,312	2,186,892	172,529	2,559,225	3,843,725	7,372,380	4,907,003
droplet	18,694	3,376	1,109,505	130,858	1,457,592	1,653,393	2,805,247	1,785,875
electrolytes_reactivity	66,967	18,063	874,449	468,769	887,548	986,445	2,340,553	1,953,147
electrolytes_redox	46,069	33,579	4,017,583	322,483	4,945,260	7,442,261	11,512,752	9,064,929
electrolytes_scaled_sep	30,768	21,186	2,307,396	215,369	2,847,853	11,683,800	8,801,619	5,116,940
geom_orca6	199,805	36,054	10,321,888	1,398,634	11,579,867	18,310,972	35,343,998	22,839,614
low_spin_23	12,939	12,045	927,730	90,573	1,113,772	2,194,086	3,228,506	2,213,493
ml_elytes	51,560	35,054	7,608,642	360,904	9,672,793	15,638,665	22,494,066	13,464,794
ml_mo	77,603	40,658	6,098,677	543,153	7,336,479	12,931,862	20,795,278	17,937,539
ml_protein_interface	61,856	35,118	8,314,895	432,979	9,480,078	16,584,221	26,101,394	15,494,518
mo_hydrides	3,396	2,870	232,942	23,772	280,311	664,784	851,813	781,762
nakb	5,606	1,188	651,552	50,642	777,349	1,297,268	2,280,331	1,551,414
noble_gas	1,915	136	66,993	13,651	84,800	43,394	116,497	87,580
noble_gas_compounds	18	12	189	126	179	326	560	518
omol	1,428,279	593,876	92,071,794	9,997,947	109,334,787	170,492,089	291,413,475	218,967,571
orbnet_denali	51,102	30,339	2,305,764	357,714	2,594,095	4,028,552	7,875,696	5,188,774
pdb_fragments_300K	93,334	68,881	11,255,165	653,329	13,208,426	21,291,730	36,314,068	22,292,731
pdb_fragments_400K	92,698	68,485	11,177,181	648,881	13,058,825	20,774,760	36,149,798	22,215,370
pdb_pockets_300K	16,870	15,453	3,245,155	118,069	3,915,033	8,616,479	10,415,619	6,407,044
pdb_pockets_400K	3,204	3,092	630,613	22,422	701,104	1,658,125	2,019,313	1,222,919
pmechdb	49,832	28,171	2,951,782	379,418	3,316,002	6,028,189	9,928,066	6,572,474
protein_core	35,406	14,980	4,197,118	247,840	4,436,976	8,229,618	13,219,684	7,738,794
protein_interface	44,760	16,127	4,298,048	313,320	4,938,560	7,052,373	13,235,662	8,007,220
rgd_uks	75,316	980	1,337,124	527,212	1,340,336	1,504,737	3,810,675	3,314,638
rmechdb	2,803	731	58,528	28,024	59,403	76,928	171,675	150,328
rna	34,434	5,168	3,661,659	241,036	4,108,650	7,166,774	12,646,859	8,583,065
rpmd	18,652	2,992	1,412,713	152,365	1,721,533	2,025,287	3,893,696	2,478,985
scaled_separations_exp	53,180	37,168	5,020,630	372,260	5,549,452	5,214,145	13,616,510	8,822,261
spice	44,447	19,603	1,553,948	311,129	1,686,049	2,413,029	4,992,500	3,292,358
tm_react	158,189	86,819	9,209,119	1,107,323	10,873,610	17,179,722	29,890,848	22,389,065
translx	213,731	171	2,974,386	1,496,117	2,977,381	3,345,716	8,459,481	7,799,805
TOTAL	3,985,130	1,302,074	218,519,775	30,203,025	253,974,706	401,137,813	688,904,594	491,385,480



H LMDB Conversion Quality Audit

Table ?? reports raw failure counts per vertical from the post-conversion audit (lmdb_status_audit.py). *QTAIM fail*: records where the qtaim.lmdb entry is empty, malformed, or unpickleable. *ORCA fail*: same for orca.lmdb (includes SCF non-convergence flagged at the parse stage). *Other fail*: failures across structure, charge, bond, fuzzy, and other LMDBs (all zero in this release). *Max drop*: largest number of structure.lmdb keys absent from any single descriptor LMDB; values ≤ 4 across all verticals.

Vertical	<i>N</i>	QTAIM fail	ORCA fail	Other fail	Max drop
omol	1,429,192	17	896	0	1
anilxbb_reparse	737,210	2	0	0	0
ani2x	215,642	—	—	—	—
trans1x	213,731	—	—	—	—
geom_orca6	199,805	—	—	—	—
tm_react	158,190	0	1	0	0
pdb_fragments_300K	93,334	0	0	0	1
pdb_fragments_400K	92,698	—	—	—	—
ml_mo	77,603	1	23	0	0
rgd_uks	75,316	—	—	—	—
electrolytes_reactivity	66,974	0	7	0	0
ml_protein_interface	61,859	0	3	0	0
scaled_separations_exp	53,219	2	37	0	0
ml_elytes	51,599	1	38	0	0
orbnet_denali	51,102	—	—	—	—
pmechdb	49,832	—	—	—	—
electrolytes_redox	46,081	0	12	0	0
protein_interface	44,972	0	212	0	0
spice	44,450	1	2	0	4
protein_core	35,406	—	—	—	—
rna	34,745	0	311	0	0
electrolytes_scaled_sep	30,769	0	1	0	0
dna	24,647	—	—	—	—
droplet	18,694	—	—	—	—
rpmd	18,652	0	0	0	1
pdb_pockets_300K	16,927	0	57	0	0
5A_elytes	14,202	0	2	0	2
low_spin_23	12,939	—	—	—	—
nakb	5,606	—	—	—	—
mo_hydrides	3,396	—	—	—	—
pdb_pockets_400K	3,210	1	5	0	0
rmechdb	2,803	—	—	—	—
noble_gas	1,915	—	—	—	—
noble_gas_compounds	18	—	—	—	—
Total	3,986,738	25	1607	0	4

Figure 6 shows per-method descriptor coverage across all 34 verticals. Rows are descriptor methods grouped by family; columns are verticals ordered by structure count (largest left). Universal methods (Hirshfeld, ADCH, CM5, Becke charges; Fuzzy bond orders; ORCA Mayer/Löwdin/SCF/gradient/dipole) reach $\geq 99\%$ on every vertical. QTAIM critical-point records achieve $\geq 99.9\%$ coverage.

I LMDB Conversion Quality Audit

Table ?? reports raw failure counts per vertical from the post-conversion audit (lmdb_status_audit.py). *QTAIM fail*: records where the qtaim.lmdb entry is empty, malformed, or unpickleable. *ORCA fail*: same for orca.lmdb (includes SCF non-convergence flagged at the parse stage). *Other fail*: failures across structure, charge, bond, fuzzy, and other LMDBs (all zero in this release). *Max drop*: largest number of structure.lmdb keys absent from any single descriptor LMDB; values ≤ 4 across all verticals.

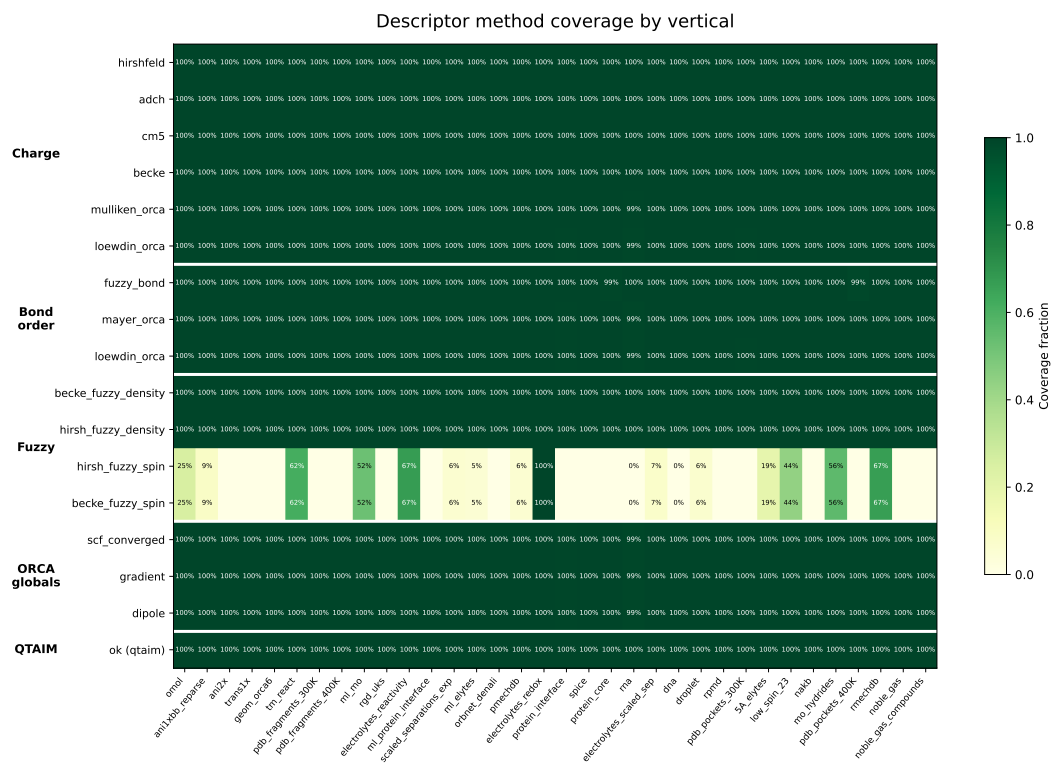


Figure 5: Per-method descriptor coverage fraction across all 34 verticals. Green = full coverage; white = absent. Vertical white lines separate descriptor families. The single ok (qtaim) column reports qtaim.lmdb record validity.

Vertical	<i>N</i>	QTAIM fail	ORCA fail	Other fail	Max drop
omol	1,429,192		17	896	0
anilxbb_reparse	737,210		2	0	0
ani2x	215,642		—	—	—
trans1x	213,731		—	—	—
geom_orca6	199,805		—	—	—
tm_react	158,190		0	1	0
pdb_fragments_300K	93,334		0	0	1
pdb_fragments_400K	92,698		—	—	—
ml_mo	77,603		1	23	0
rgd_uks	75,316		—	—	—
electrolytes_reactivity	66,974		0	7	0
ml_protein_interface	61,859		0	3	0
scaled_separations_exp	53,219		2	37	0
ml_elytes	51,599		1	38	0
orbnet_denali	51,102		—	—	—
pmechdb	49,832		—	—	—
electrolytes_redox	46,081		0	12	0
protein_interface	44,972		0	212	0
spice	44,450		1	2	4
protein_core	35,406		—	—	—
rna	34,745		0	311	0
electrolytes_scaled_sep	30,769		0	1	0
dna	24,647		—	—	—
droplet	18,694		—	—	—
rpmd	18,652		0	0	1
pdb_pockets_300K	16,927		0	57	0
5A_elytes	14,202		0	2	2
low_spin_23	12,939		—	—	—
nakb	5,606	16	—	—	—
mo_hydrides	3,396		—	—	—
pdb_pockets_400K	3,210		1	5	0
rmechdb	2,803		—	—	—
noble_gas	1,915		—	—	—

Figure 6 shows per-method descriptor coverage across all 34 verticals. Rows are descriptor methods grouped by family; columns are verticals ordered by structure count (largest left). Universal methods (Hirshfeld, ADCH, CM5, Becke charges; Fuzzy bond orders; ORCA Mayer/Löwdin/SCF/gradient/dipole) reach $\geq 99\%$ on every vertical. QTAIM critical-point records achieve $\geq 99.9\%$ coverage.

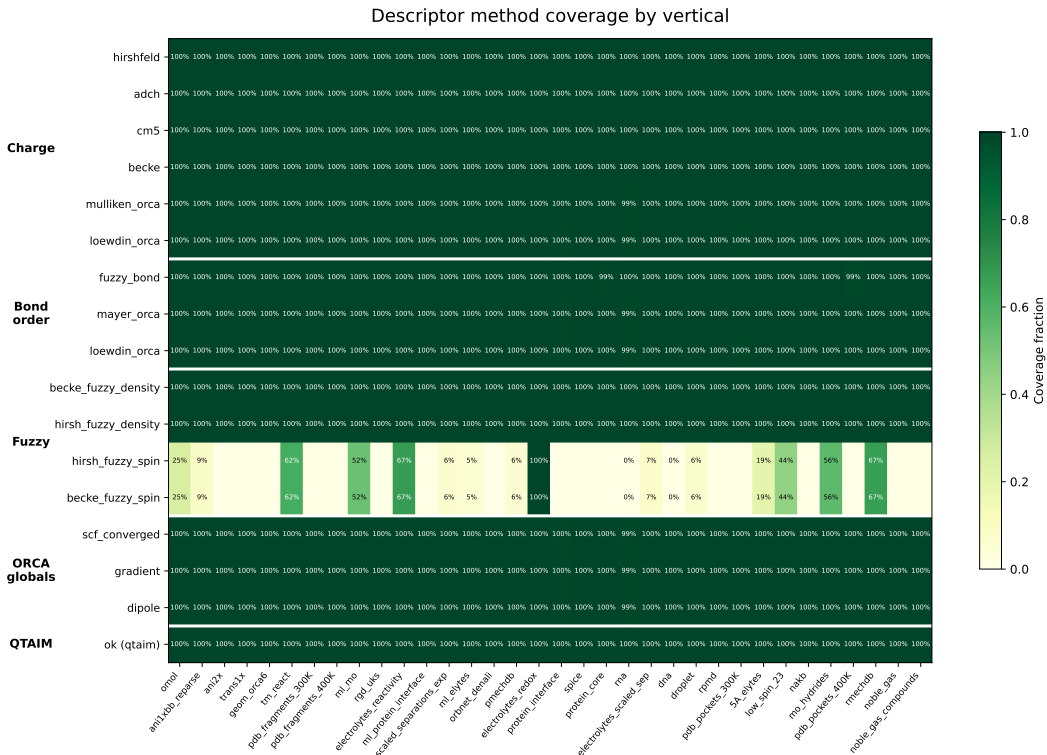


Figure 6: Per-method descriptor coverage fraction across all 34 verticals. Green = full coverage; white = absent. Vertical white lines separate descriptor families. The single ok (qtaim) column reports qtaim.lmdb record validity.

481 J Extended Bond-Order Histograms

482 K Additional Cross-Method Residual Plots

483 L Manifest Schema Documentation

484 M Code Structure Overview

485 P